The Banksys

Build an AI art-learning assistant in Python + Gradio

Student tutorial: follow the steps in order and build the project as you read.

Contents

Welcome: what you are building 3

How to use the hints 3

How to use this tutorial 3

The roadmap 3

Before you start 5

Set up your project 5

Demo mode (no API key needed) 6

The big picture 6

Build it, step by step 7

Step 1: Run the starter and read how it works 7

Step 2: Feature 1, look up an art style 9

Step 3: Feature 2, report the size of the upload 11

Step 4: Feature 3, instructions for each improvement area 12

Step 5: Feature 4, build the AI prompt 14

Step 6: Feature 5, format the AI's answer 16

Step 7: Turn on the real AI and test everything 19

Step 8: Optional extensions 20

What you learned 20

Welcome: what you are building

The Banksys is a small web app that helps you learn about art. You will be able to:

1. Browse art styles (Impressionism, Cubism, Pop Art, Realism) and read about each one.
2. Upload your own drawing or painting.
3. Pick a **target style** and something you want to improve (shading, color, ...).
4. Get a friendly critique from an AI art teacher, split into **Strengths**, **Areas to Improve** and **Suggested Exercises**.

The folder you were given is a **scaffold**. The app already launches, but the important functions are empty and marked `TODO`. This tutorial walks you through every `TODO`, in the order you should do them. It does **not** hand you the answers. Instead, for each one you will find:

- **Recall**: a reminder of the Python idea you need, with a small example on a different topic.
- **Your task**: what the function has to do, in plain English.
- **Hints**: nudges that get more specific, so you can reveal only as much help as you need.
- **Check it**: how to run it and what you should see when it works.

> **TIP**
> Type everything yourself. The fastest way to learn is to try something, read the error message, and fix it.

How to use the hints

1. **Try first.** Read the task and attempt it on your own for about **10 minutes**.
2. Still stuck? Read **Hint 1** only. Try again.
3. Still stuck? Read **Hint 2**, then **Hint 3** if there is one. Each hint gives away a little more.
4. Still stuck after all the hints? Ask a classmate or an instructor. Describe what you tried and what error you saw.

Struggling for a few minutes is normal, and it is how the idea sticks. Asking for help is fine, but try the hints first.

How to use this tutorial

- Keep this PDF open next to your editor. Work through the steps **in order**. Each step builds on the last.
- Whenever you see **Checkpoint**, stop and run the app before moving on.
- Boxes labelled **Stuck?** list the error messages students see most often and what they mean.

> **NOTE**
> Instructor note: a separate teacher answer key with full solutions exists in the `answer_key/` folder.

The roadmap

Step	File	What you build
1	`app.py`	Run the starter app and read how it works (no coding)
2	`art_styles.py`	Feature 1: look up an art style's description
3	`app.py`	Feature 2: report the size of an uploaded image
4	`feedback.py`	Feature 3: instructions for each improvement area
5	`feedback.py`	Feature 4: build the prompt that goes to the AI
6	`feedback.py`	Feature 5: format the AI's answer into three sections
7	(your `.env`)	Turn on the real AI and test the whole app
8	any	Optional extensions

Before you start

Set up your project

Open a terminal inside the `the_banksys` folder, then run the commands for your computer **one at a time**.

Open the terminal in the right folder:

- **Mac:** open the **Terminal** app, type `cd ` (with a space), drag the `the_banksys` folder onto the window, and press Enter. (Or right-click the folder in Finder and choose **New Terminal at Folder**.)
- **Windows:** open the `the_banksys` folder in File Explorer, click the address bar at the top, type `powershell`, and press Enter. A PowerShell window opens already inside the folder.
- **Either:** if you use VS Code, open the folder and choose **Terminal > New Terminal**.

On a Mac:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python app.py
```

On Windows (PowerShell or Command Prompt):

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python app.py
```

What each line does:

- `python -m venv venv` creates a private "box" of Python packages just for this project. (On a Mac the command is `python3` because many Macs have no plain `python` until the box is on.)
- `source venv/bin/activate` (Mac) or `venv\Scripts\activate` (Windows) steps into that box. You should now see `(venv)` at the start of your terminal line. **Do this every time you open a new terminal.**
- `pip install -r requirements.txt` installs the four packages the app needs (Gradio for the web page, Google GenAI for the AI, python-dotenv for reading your key, Pillow for images).
- `cp .env.example .env` (Mac) or `copy .env.example .env` (Windows) makes your private settings file. You can leave the key blank for now.
- `python app.py` starts the app. Open the address it prints (usually `http://127.0.0.1:7860`).

> **STUCK?**
> **Windows:** if `python` is "not found", try `py -m venv venv` instead (then use plain `python` once the box is on). If PowerShell says *running scripts is disabled on this system* when you activate, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer **Y**, and try again. Or just use **Command Prompt** instead of PowerShell. **Mac:** if `python3` is not found, install Python from python.org.

To stop the app, click the terminal and press `Ctrl + C`. **After you edit a Python file you must stop and restart the app** to see your change.

Demo mode (no API key needed)

The AI part needs a Google Gemini API key, which your teacher may give you later (or you can create one at Google AI Studio, aistudio.google.com/apikey). **Without a key the app runs in demo mode.** Everything works except the AI never looks at your picture; instead it prints a clearly labelled message plus **the prompt your code built**. That is perfect for checking your work, so you can finish Steps 1 to 6 without any key.

> **WARNING**
> Never paste your API key into a Python file, share it, or commit `.env` to GitHub. Keys go only in the `.env` file.

The big picture

When you click a button, information travels like this:

```
you click / upload in the browser
         |
         v
  app.py  (Gradio calls one of its functions)
         |
         +--> art_styles.py   look up a style description
         |
         +--> feedback.py     choose instructions -> build the prompt
                    |
                    v
               ai_helper.py    sends image + prompt to the AI, gets text back
                    |
                    v
               feedback.py     format the text nicely
                    |
                    v
  app.py shows the result in the browser
```

File	Job
`app.py`	The screen: tabs, buttons, and the functions that run when you click
`art_styles.py`	The dictionary of art styles and the lookup function
`feedback.py`	Turns your choices into a prompt, and the AI's answer into readable sections
`ai_helper.py`	The only file that talks to the AI. **You never need to edit it.**

You will edit `art_styles.py`, `feedback.py`, and one small piece of `app.py`.

Build it, step by step

Step 1: Run the starter and read how it works

Goal: understand the code that is already written, before you add to it. There is nothing to type in this step.

1a. Click around

Run `python app.py` and open the page. There are three tabs:

1. **Learn About Styles**: pick a style. Right now the description box shows a TODO message. That is your first job.
2. **Upload Artwork**: drop in any image. It says "Artwork received!" but nothing else.
3. **Get Feedback**: pick a style and a focus area and click the button. You will see a demo-mode message.

1b. Read `app.py`

Open `app.py` and find the three parts of every Gradio app.

Part 1: handler functions. These are ordinary Python functions. Gradio calls them when something happens in the browser. Here is the first one:

```
def show_style(style_name):
    return get_style_description(style_name)
```

Gradio passes the dropdown's current value in as `style_name`. Whatever the function **returns** is displayed in an output component.

Part 2: components. These are the buttons, dropdowns and text boxes on the screen.

```
style_dropdown = gr.Dropdown(choices=get_style_names(), label="Choose a style")
style_output = gr.Textbox(label="Description", lines=4)
```

Part 3: wiring. This line connects them:

```
style_dropdown.change(show_style, inputs=style_dropdown, outputs=style_output)
```

Read it aloud: *"When the dropdown changes, call* `show_style`*, give it the dropdown's value, and put the result in* `style_output`*."* Every interactive thing in this app is built from that one pattern: **event, function, inputs, outputs.**

> **CONCEPT**
> The components listed in `inputs=[...]` are matched to your function's parameters by **position**, not by name. In `critique_button.click(handle_critique, inputs=[image_input, target_style, area_choice], ...)`, the first input becomes the first parameter `image`, the second becomes `style`, and the third becomes `area`. If you ever mix up the order, your function receives the wrong thing.

1c. Read `feedback.py`'s `get_critique`

This function is already finished and shows how the pieces connect:

```
def get_critique(image, style, area):
    instructions = get_improvement_instructions(area)      # Feature 3 (you write)
    prompt = build_prompt(style, area, instructions)       # Feature 4 (you write)
    ai_text, is_real = ask_ai_about_image(image, prompt)   # ai_helper.py (done)

    if not is_real:
        return ai_text
    return format_feedback(ai_text)                        # Feature 5 (you write)
```

It runs **four stages in a row**, and each stage's output feeds the next. Keep this picture in mind. Steps 4, 5 and 6 each build one stage.

> **CHECKPOINT**
> Can you answer these? (1) What does a handler function return to? (2) Which file talks to the AI? (3) Which three functions in `feedback.py` are still unfinished? If yes, move on.

Step 2: Feature 1, look up an art style

File: `art_styles.py` **Function:** `get_style_description(style_name)`

Goal: when the user picks a style in the dropdown, show its description.

Ideas you need: dictionaries, `if` statements, returning a value.

Understand the data first

Look at the top of `art_styles.py`. `ART_STYLES` is a **dictionary**: it stores pairs of *key* and *value*. Here the keys are style names and the values are descriptions. The helper `get_style_names()` (already written) returns just the keys, which is how the dropdown gets its choices.

TODO 1: handle "nothing chosen yet"

Recall: an `if` statement runs some code only when a condition is true. In Python, some values count as "empty" or *falsy*: `None`, `""` (an empty string), `0` and `[]`. The word `not` flips a condition. A generic example on a different topic:

```
def pizza_message(topping):
    if not topping:
        return "No topping picked."
    return "Great choice!"
```

Your task: when the page first loads, or the user clears the dropdown, `style_name` is `None` or an empty string. In that case the function should return a friendly message asking the user to choose a style, and stop there.

- **Hint 1:** this is an `if` at the very top of the function. You want to catch the case where the argument has no real value.
- **Hint 2:** you don't need to write two checks (one for `None`, one for `""`). Think about which single condition is true for both, like the pizza example.
- **Hint 3:** in words: *if the style name is empty, return a polite "please choose a style" message.*

TODO 2: look up the style

Recall: a dictionary maps keys to values. There are two common ways to read one:

```
prices = {"slice": 3, "whole pie": 20}
prices["slice"]              # gives 3
prices.get("calzone", 0)     # gives 0, because "calzone" is not a key
```

Your task: find the description that belongs to `style_name` in `ART_STYLES`.

The scaffold lists three options (A: square brackets, B: `.get`, C: an if/elif chain). Before you choose, think about these questions:

- What happens with `prices["calzone"]` when the key is missing? Try it in a Python shell.
- What does `.get()` do with its **second** argument?
- Which option would you have to edit every time you add a new art style?
- **Hint 1:** option B is the shortest and safest. It never crashes on a missing key.
- **Hint 2:** the second argument of `.get()` is what gets returned when the key is not found. Pick a short message like "Unknown style".

TODO 3: return it

Your task: make sure the function **returns** the description (not `print` s it). Gradio only displays what a function returns. You can do TODO 2 and TODO 3 on one line. Remove the old `TODO` return line at the bottom, since it would otherwise hide your work.

CHECKPOINT

Restart the app. In tab 1 choose each style and read the description. Then test your function without the browser. In the terminal, run: `python -c "from art_styles import get_style_description as g; print(g('Pop Art')); print(g(None)); print(g('Nope'))"` You should see the Pop Art description about bold flat color, then your "please choose" message, then your "unknown style" message.

STUCK?

If you still see the TODO message, you either did not save the file or did not restart the app. `IndentationError` means the lines inside the function are not indented by four spaces. `KeyError` means you used square brackets on a key that is not in the dictionary.

Step 3: Feature 2, report the size of the upload

File: `app.py` **Function:** `handle_upload(image)`

Goal: when the user uploads artwork, tell them its size in pixels.

Ideas you need: objects and attributes, f-strings.

How the image travels

Read the comment block at the top of `handle_upload`. The component is `gr.Image(type="pil", ...)`. That `type="pil"` setting tells Gradio to hand your function a **PIL Image object** (PIL is the Pillow library). If nothing is uploaded, `image` is `None`. That is why the function checks `if image is None` first. **Always check for `None` before using an input**, or your code will crash trying to read a property of nothing.

TODO 1: add the size

Recall: an **object** can have **attributes**, which you read with a dot. An **f-string** is a string that starts with `f` and lets you put variables inside `{ }`. A generic example:

```
class Dog:
    age = 4
d = Dog()
print(f"Dog is {d.age} years old")
```

Your task: change the success message so it also tells the user how wide and how tall the image is, in pixels. The scaffold's own comment tells you the two attribute names to use.

- **Hint 1:** read the `TODO 1` comment inside `handle_upload`. It names the attributes.
- **Hint 2:** use an f-string and put the two attributes inside `{ }`, with a short word or symbol between them.

> **CHECKPOINT**
> Restart, go to tab 2, and upload any picture. The Status box should show the width and height. Check that the numbers match your file's properties (on a Mac, Get Info; on Windows, Properties). Then click the small **x** to remove the image; you should see the "No artwork uploaded yet." message.

> **NOTE**
> The upload component `image_input` is defined in tab 2 but **used in tab 3** (in the `inputs=[image_input, ...]` list). Components are just Python variables, so any tab can use them.

Step 4: Feature 3, instructions for each improvement area

File: `feedback.py` **Function:** `get_improvement_instructions(area)`

Goal: the user picks one of five areas (`IMPROVEMENT_AREAS`). Return **one sentence** that tells the AI what to focus on for that area.

Ideas you need: `if / elif / else` and comparing strings with `==` .

Why is this a separate function?

A good prompt is specific. "Give feedback" gets vague answers. "Focus on how light and shadow create form and depth" gets useful ones. This function holds the specific sentence for each area so `build_prompt` (next step) can simply plug it in.

TODO 1 and TODO 2: one branch per area

Recall: `if / elif / else` picks exactly one branch. `==` **compares** two values, while a single `=` **assigns**. A generic example:

```
def drink_advice(size):
    if size == "small":
        return "Good for one person."
    elif size == "large":
        return "Good for sharing."
    else:
        return "Ask the staff."
```

Your task: the starter text has you write branches for each area. Add an `elif` for **Composition**, **Color** and **Proportions**, each returning its own sentence for the AI. Ask yourself, for each area: *what would a real art teacher look at?*

- **Composition:** think about arrangement, balance and where the eye is drawn.
- **Color:** think about harmony, contrast and mood.
- **Proportions:** think about the relative sizes of shapes and body parts.

Write the sentences in your own words. They only need to be **different and specific**.

- **Hint 1:** the starter shows one branch for Shading as a model. Copy its shape, not its words.
- **Hint 2:** each `if` , `elif` and `else` line ends with a colon, and the `return` under it is indented.
- **Hint 3:** in words: *if area is Shading, return sentence A; otherwise if it is Composition, return sentence B; ...*

TODO 3: the `else`

Recall: `else` has no condition. It catches everything that did not match an earlier branch.

Your task: use `else` for "General feedback" and for anything unexpected, and return a sentence that covers the **whole piece**. Why is it safer to use `else` here than to write `elif area == "General feedback"` ? (Think about what happens if a typo slips in.)

CHECKPOINT

In the terminal run `python -c "from feedback import get_improvement_instructions as g; print(g('Color')); print(g('General feedback'))"` and check you get two different sentences. Then try `g('Banana')` and confirm it falls into the `else` without crashing.

STUCK?

`SyntaxError: invalid syntax` on an `if` line usually means a missing colon `:` at the end. `SyntaxError` near an `elif` often means it is not lined up with its `if`. All of `if`, `elif` and `else` must start at the same indent.

Step 5: Feature 4, build the AI prompt

File: `feedback.py` **Function:** `build_prompt(style, area, instructions)`

Goal: assemble one string, the **prompt**, that tells the AI who it is, what the student wants, and exactly how to format its answer.

Ideas you need: string concatenation, f-strings, and `\n` (a newline character).

What is a prompt?

The AI knows nothing about your student except what you write in the prompt. Everything it needs (the role, the target style, the focus area, the answer format) must be in that one string. Building good prompts is a real skill, and it is mostly careful writing.

Recall: `\n` means "new line" and `\n\n` leaves a blank line. `+=` adds onto the end of an existing string. An f-string drops variables into the text. A generic example:

```
note = "You are a friendly pizza chef.\n"
note += f"The customer ordered a {size} pizza.\n"
note += "Keep the answer short."
```

TODO 1 to TODO 3: build the prompt up in pieces

Your task: start a string variable and add pieces onto it with `+=`. A good prompt for this app includes all of these, in an order that reads naturally:

- [ ] **A role** for the AI (for example, what kind of teacher it is and what tone it should use).
- [ ] **The target style** the student wants to move toward, using the `style` parameter.
- [ ] **The improvement area** they chose, using the `area` parameter.
- [ ] **The focus sentence** from Step 4, which arrives in the `instructions` parameter.

Write the wording yourself. Make it sound like a real person talking to a teacher.

- **Hint 1:** begin with something like `prompt = "..."`, then add lines with `prompt += ...`.
- **Hint 2:** to include a variable in the middle of a sentence, start the string with `f` and put the variable name in `{ }`.
- **Hint 3:** remember the `\n` at the end of each piece, or every sentence will run together in one long line.

TODO 4: ask for three labelled sections

Why this matters: in Step 6, your code will **split the AI's answer on these exact labels**. The labels must be spelled exactly like this, with a colon:

- `STRENGTHS:`
- `AREAS TO IMPROVE:`
- `EXERCISES:`

Your task: add text that asks the AI to reply in exactly three labelled sections, in that order, using those exact labels. You may also ask it to keep each section short and to use bullet points.

- **Hint 1:** the labels must match character for character, including capitals and the colon.
- **Hint 2:** a long string can be split over several lines inside parentheses, with a quoted piece on each line and **no commas** between them. Python joins them.

TODO 5: return the prompt

Your task: return the finished string. Which variable holds it?

> **NOTE**
> **Checkpoint (demo mode):** Restart the app. Upload an image in tab 2. In tab 3 choose a style and an area, then click **Get feedback**. In demo mode the output shows your prompt at the bottom. Read it as if you were the AI. Tick off the checklist above: does it have a role, the style, the area, the focus sentence, and the three exact labels? Change the style and area and click again. The prompt text should change to match.

> **WHY**
> Reading the prompt in demo mode is like seeing what the AI sees. If something is missing from the prompt, the AI cannot possibly use it. Always debug prompts by printing them first.

> **STUCK?**
> If you see the literal text `{style}` in the output, you forgot the `f` before the opening quote. If a piece of the prompt is missing, check that you used `+=` and not `=`, which would throw away what you built before. If the function returns nothing useful, did you add the `return` line?

Step 6: Feature 5, format the AI's answer

File: `feedback.py` **Function:** `format_feedback(ai_text)`

Goal: the AI answers with the three labelled sections. Split them apart and display them as clean headings and lists.

Ideas you need: `str.split` , `str.strip` , `in` , unpacking, `if` .

What the AI text looks like

Because of your prompt, the reply should look something like this:

```
STRENGTHS:
- Nice use of contrast in the sky
AREAS TO IMPROVE:
- The hand looks too small compared with the head
EXERCISES:
1. Draw 10 hands from photos, 2 minutes each
```

And this is what we want the user to see:

```
## Strengths
- Nice use of contrast in the sky

## Areas to Improve
- The hand looks too small compared with the head

## Suggested Exercises
1. Draw 10 hands from photos, 2 minutes each
```

(`##` makes a heading, because the output box is a Markdown component. You can add emoji to the headings like the target in the starter file if you wish.)

TODO 1: split into three pieces

Recall: `text.split(separator, 1)` cuts a string at the **first** occurrence of the separator and gives back a list of two parts. You can **unpack** a two-item list into two variables in one line. A generic example:

```
order = "TOPPING: cheese SIZE: large"
left, right = order.split("SIZE:", 1)
# left  is "TOPPING: cheese "
# right is " large"
```

Your task: cut the AI's text at each label so you end up with three separate pieces: the strengths, the improvements and the exercises.

- **Hint 1:** you are cutting at three labels, so you will split more than once. Each cut can use whatever is left over from the cut before.
- **Hint 2:** the first cut is at `"STRENGTHS:"` . Anything the AI wrote *before* that label ends up in the left variable. You can simply ignore it.
- **Hint 3:** in words: *cut at STRENGTHS: and keep the right side; cut that at AREAS TO IMPROVE: (left is the strengths); cut the remainder at EXERCISES: (left is the improvements, right is the exercises).*

TODO 2: strip extra whitespace

Recall: `.strip()` removes spaces and blank lines from both ends of a string. In the pizza example, `left.strip()` would give `"TOPPING: cheese"` with no trailing space.

Your task: clean each of your three pieces so there are no stray blank lines at the start or end.

TODO 3: build the output, and handle a missing section

Your task, part A: build one string that has a heading line before each piece, in the order Strengths, Areas to Improve, Suggested Exercises, with a blank line between sections. Return it.

- **Hint 1:** use `+` or an f-string to join the headings and your cleaned pieces. Remember `\n` for new lines.

Your task, part B (the bonus in the starter file): *what if the AI forgot a section?* If a label is missing, `split` returns only **one** part, and unpacking it into two variables crashes with a `ValueError`. So check **before** you split. If any label is missing, return the AI's text unchanged.

- **Hint 1:** recall the word `in`: `"SIZE:" in order` is `True` or `False`. Put `not` in front to flip it.
- **Hint 2:** you need to check three labels. `or` is true when at least one of its conditions is true.
- **Hint 3:** in words: *if the text is missing STRENGTHS: or is missing AREAS TO IMPROVE: or is missing EXERCISES: -> return the text as it is.* Put this check **before** the splitting.

Test it without any API key

You can test `format_feedback` on its own by giving it fake AI text. Run this in the terminal:

```
python -c "from feedback import format_feedback as f; print(f('STRENGTHS:\n- good line\nAREAS TO
    IMPROVE:\n- hands\nEXERCISES:\n1. draw hands'))"
```

You should see three headed sections, each with its bullet underneath. Then try `f('The AI ignored my format')` and check you get the same sentence back untouched. Good code deals with surprises gracefully.

Bonus challenge: when the AI uses bold headings

Models often write `**STRENGTHS:**` in bold. Try it:

```
python -c "from feedback import format_feedback as f; print(repr(f('**STRENGTHS:**\n-
    a\n\n**AREAS TO IMPROVE:**\n- b\n\n**EXERCISES:**\n1. c')))"
```

Look at the output. Where did the stray `**` characters end up? Can you write a small helper function that removes them from each piece? Think about which string method removes unwanted characters from the ends of a string. (Look up what `.strip()` does when you give it an argument.)

> **CHECKPOINT**
> There is nothing new to see in demo mode, because demo and error messages are shown as-is (look at the `if not is_real` line in `get_critique`). You will see your formatting in real use in the next step. Make sure the terminal tests above passed first.

STUCK?

`ValueError: not enough values to unpack` means a label was missing from the text before you split. Add a check that runs first. If sections run together, check that you added blank lines between them. If the heading text is missing entirely, check you joined all six pieces (three headings, three sections).

Step 7: Turn on the real AI and test everything

Goal: see your whole pipeline working with real feedback.

1. Get a Gemini API key from your teacher (or create one at aistudio.google.com/apikey).
2. Open `.env` (the file you copied from `.env.example`) and set the key with no quotes and no spaces:

```
GEMINI_API_KEY=your-key-here
GEMINI_MODEL=gemini-3.5-flash
```

3. Stop the app (`Ctrl + C`) and run `python app.py` again. The yellow demo-mode banner at the top should be **gone**.
4. Upload a real drawing, choose a target style and an area, and click **Get feedback**. This takes a few seconds.

You should get a formatted critique with **Strengths**, **Areas to Improve** and **Suggested Exercises**.

Run your final test plan

Test	What you should see
Pick each of the 4 styles in tab 1	Four different descriptions
Upload a picture, then check tab 2	Its width and height
Click Get feedback with no picture	A message asking you to upload artwork first
Get feedback with no style chosen	A message asking you to choose a target style
Same picture, 3 different areas	Noticeably different advice each time
Same picture, 2 different target styles	Advice that mentions each style

STUCK?
If you see `The AI request failed: ...`, read the end of the message. `API key not valid` or `403` means the key is wrong or has a space in it. `model` errors mean `GEMINI_MODEL` has a typo. If the banner still shows, `.env` must be in the same folder as `app.py` and the app must have been restarted.

Step 8: Optional extensions

Finished early? Each of these is a real feature you can add. Do them in any order, and use the same method as before: think, try, then look for hints in the docs or ask.

1. **More styles.** Add Surrealism, Ukiyo-e, Art Nouveau or your own favorites to `ART_STYLES`. Nothing else needs to change, because the dropdown reads the dictionary's keys. Why?
2. **Richer styles.** Turn each value into a dictionary such as `{"description": "...", "artists": ["Monet", "Renoir"]}`. Then update `get_style_description()` to show the artists too. (What does `ART_STYLES.get(...)` return now?)
3. **Image details.** Show the image format and mode in `handle_upload`. Look up the attributes of a PIL `Image`.
4. **Skill level.** Add a `gr.Radio` for beginner or advanced in `app.py`, pass it through `handle_critique` and `get_critique`, and add a sentence to the prompt about the student's level.
5. **History.** Use a `gr.State([])` to keep a list of past critiques and show them in a new tab.
6. **Compare two drawings.** A new tab with two `gr.Image` inputs and a prompt that asks the AI to compare them.

What you learned

- The **event, function, inputs, outputs** pattern that every Gradio app is built on.
- **Dictionaries** and safe lookups with `.get()`.
- `if / elif / else` to make decisions on text.
- **String building** with f-strings, `+=` and `\n`.
- **Splitting and cleaning text** with `split`, `strip` and `in`.
- That a **prompt is code**: precise instructions produce precise results, and you can test one with no AI at all.